

```
In [2]: # Install required libraries (uncomment if needed)
# !pip install pandas numpy matplotlib scikit-learn

# Import libraries with standard aliases
import pandas as pd import numpy as np
import matplotlib.pyplot as plt from
sklearn import datasets import warnings
warnings.filterwarnings('ignore') # Hide warning messages for cleaner output

print("☑ All libraries imported successfully!")
print(f"Pandas version: {pd.__version__}") print(f"NumPy
version: {np.__version__}")
```

Matplotlib is building the font cache; this may take a moment.

☑ All libraries imported successfully!

Pandas version: 3.0.6

NumPy version: 2.5.3

```
In [ ]: # Load a simple dataset (Iris flowers - a classic beginner dataset) from
sklearn.datasets import load_iris
# Load the data iris = load_iris() print("Dataset loaded successfully!")
print(f"Dataset shape: {iris.data.shape}") # This will print the number of s
print(f"Features: {iris.feature_names}") # This will print the names of the
print(f"Target classes: {iris.target_names}") # This will print the names of
```

Dataset loaded successfully!

Dataset shape: (150, 4)

Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)'] Target classes: ['setosa' 'versicolor' 'virginica'] In []:

```
# Convert to pandas DataFrame for easier handling df =
pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target_names[iris.target]

# Display first few rows
print("First 5 rows of our dataset:") print(df.head())

print("\nDataset info:") print(df.info()) # This will print a summary of the
DataFrame, including the
```

First 5 rows of our dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

```

species 0
setosa
1 setosa
2 setosa
3 setosa
4 setosa

```

Dataset info:

```
<class 'pandas.DataFrame'>
```

RangeIndex: 150 entries, 0 to 149 Data

columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	species	150 non-null	str

dtypes: float64(4), str(1)
memory usage: 6.0 KB None

In []:

```

# Create a simple scatter plot plt.figure(figsize=(10,
6))

# Plot sepal length vs sepal width, colored by species
species_colors = {'setosa': 'red', 'versicolor': 'blue', 'virginica': 'green'}

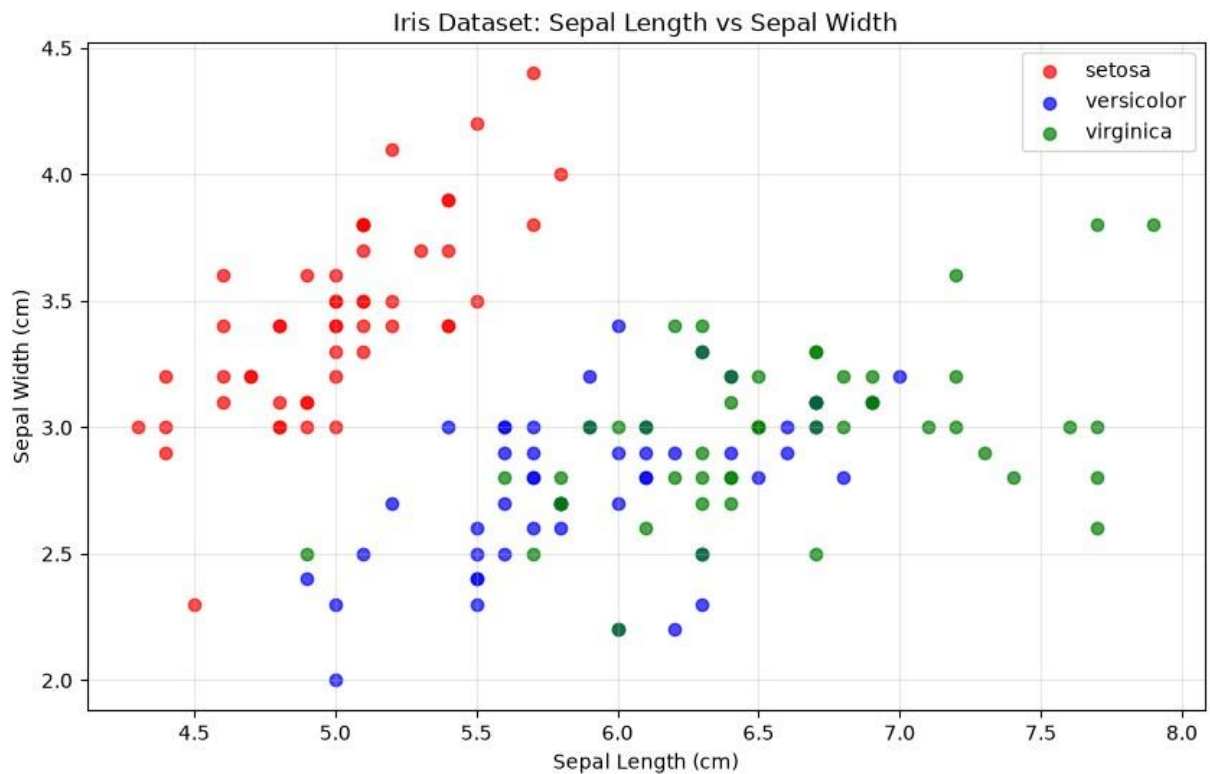
for species in df['species'].unique(): # Loop through each unique species in
species_data = df[df['species'] == species] # pandas DataFrame filtering
plt.scatter(species_data['sepal length (cm)'], # It sets the x-axis value
species_data['sepal width (cm)'], # sets the y-axis values f
c=species_colors[species], label=species,
alpha=0.7)

plt.xlabel('Sepal Length (cm)') # sets the x-axis label plt.ylabel('Sepal
Width (cm)') #sets the y-axis label plt.title('Iris Dataset: Sepal Length vs
Sepal Width') #sets the title of the plot plt.legend() #adds a legend to the plot

plt.grid(True, alpha=0.3) #sets a grid for better readability plt.show()

print("🎉 Congratulations! You've created your first data visualization!")

```



🎉 Congratulations! You've created your first data visualization!

```
In [ ]: # Basic statistical analysis
print("Basic Statistics for Iris Dataset:") print("="
★ 40)

# Calculate mean values for each species
species_means = df.groupby('species').mean() # numpy used for mean calculati
print("\nMean values by species:") print(species_means)

# Count samples per species
species_counts = df['species'].value_counts() # pandas function to count occ
print("\nSamples per species:") print(species_counts)
```

Basic Statistics for Iris Dataset:

=====

Mean values by species:

	sepal length (cm)	sepal width (cm)	petal length (cm)	\ species
setosa	5.006	3.428	1.462	
versicolor	5.936	2.770	4.260	
virginica	6.588	2.974	5.552	

	petal width (cm)	species
setosa	0.246	
versicolor	1.326	
virginica	2.026	

```
Samples per species:
species setosa      50
versicolor      50
virginica      50 Name:
count, dtype: int64
```

```
In [ ]: # Task 1: Create a simple calculation using NumPy
# Calculate the mean and standard deviation of sepal length sepal_lengths

= df['sepal length (cm)']

# Your code here:
mean_sepal_length = np.mean(sepal_lengths) # Calculate the mean of sepal len
std_sepal_length = np.std(sepal_lengths) # Calculate the standard deviation

print(f"Mean sepal length: {mean_sepal_length:.2f} cm") print(f"Standard
deviation: {std_sepal_length:.2f} cm")

# Verification (don't modify)
assert isinstance(mean_sepal_length, (float, np.floating)), "Mean should be
assert isinstance(std_sepal_length, (float, np.floating)), "Std should be a
print("☑ Task 1 completed successfully!")
```

Mean sepal length: 5.84 cm

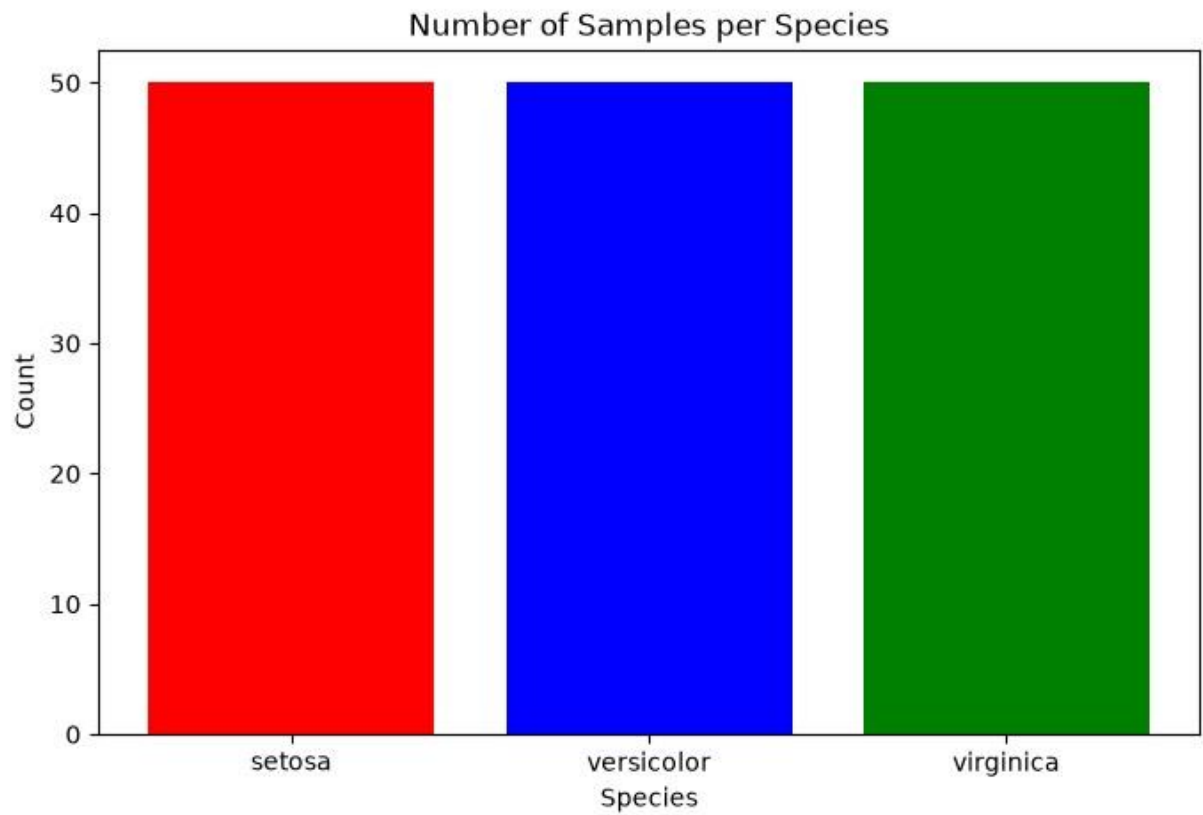
Standard deviation: 0.83 cm

☑ Task 1 completed successfully!

```
In [ ]: # Task 2: Create a simple bar chart showing species counts
species_counts = df['species'].value_counts() # Depending on the dataset, th

plt.figure(figsize=(8, 5)) # Sets the figure size
plt.bar(species_counts.index, species_counts.values, color=['red', 'blue',
plt.title('Number of Samples per Species') # Sets the title of the bar chart
plt.xlabel('Species') #Sets the x-axis label plt.ylabel('Count') #Sets the y-
axis label plt.show() # Displays the bar chart

print(f"Species distribution: {dict(species_counts)}") print("☑
Task 2 completed successfully!")
```



Species distribution: {'setosa': np.int64(50), 'versicolor': np.int64(50), 'virginica': np.int64(50)} ☒ Task 2 completed successfully!